

Dokumentacja Kodu: System Zarządzania Obecnością

Opis działania i struktury programu

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 //=====
6 //Interfejs
7 //=====
8 class IPrzedstawialny {
9     public:
10     virtual string przedstawSie() = 0;
11 };
```

Biblioteki i Interfejs (Linie 1-11)

Na początku dołączamy standardowe biblioteki do obsługi wejścia/wyjścia oraz tekstów (`string`). W linii 8 tworzymy interfejs. Zawiera on czysto wirtualną funkcję `przedstawSie()`. Dzięki temu każda klasa, która odziedziczy ten interfejs, będzie miała obowiązek stworzyć własną wersję tej funkcji, co zapewnia spójność w naszym programie.

```
13 class Osoba{
14     protected:
15     string imie;
16     string nazwisko;
17
18     public:
19     void setImie(string _imie) {
20         if(_imie.length() > 2) imie = _imie;
21     }
22     void setNazwisko(string _nazwisko) {
23         if(_nazwisko.length() > 1) nazwisko = _nazwisko;
24     }
25     string getNazwisko() { return nazwisko; }
26     virtual void drukuj() {
27         cout << " OSOBA - imie: " << imie
28              << " nazwisko: " << nazwisko;
29     }
30 };
```

Klasa Bazowa Osoba (Linie 13-29)

Ta klasa przechowuje podstawowe dane. Zmienne są oznaczone jako `protected` (linia 14), co oznacza, że są ukryte z zewnątrz, ale klasy pochodne (jak `Student`) mogą z nich swobodnie korzystać. Funkcje ustawiające (setterzy w liniach 19 i 22) mają proste zabezpieczenia – sprawdzają, czy wpisany tekst nie jest za krótki. Dodaliśmy też funkcję wirtualną `drukuj()`, która w przyszłości będzie dostosowywana przez inne klasy.

```
31 class Student : public Osoba, public IPrzedstawialny{
32     private:
33     string nr_index;
34
35     public:
36     void setNr_Index(string _nr_index) {
37         if(_nr_index.length() > 1) nr_index = _nr_index;
38     }
39     virtual void drukuj() {
40         cout << "STUDENT - imie: " << imie
41              << " nazwisko: " << nazwisko
42              << " index: " << nr_index;
43     }
44     string przedstawSie() {
45         return " STUDENT - " + imie + " " + nazwisko
46              + " index: " + nr_index;
47     }
48 };
```

Klasa Student (Linie 31-48)

Klasa ta dziedziczy cechy `Osoby` i jednocześnie wdraża nasz interfejs (linia 31). Dodaje nową zmienną – numer indeksu. W linii 39 nadpisujemy metodę `drukuj()`, by wyświetlała dane specyficzne dla studenta. W linii 44 realizujemy wymóg interfejsu, tworząc funkcję, która zwraca złączony tekst z wszystkimi danymi.

```

50 class Pracownik : public Osoba{
51     private:
52         string nr_pracownika;
53
54     public:
55     void setNr_Pracownika(string _nr_pracownika) {
56         if(_nr_pracownika.length() > 1)
57             nr_pracownika = _nr_pracownika;
58     }
59     virtual void drukuj() {
60         cout << " PRACOWNIK - imie: " << imie
61              << " nazwisko: " << nazwisko
62              << " nr: " << nr_pracownika;
63     }
64 };

```

Klasa Pracownik (Linie 50-64)

Kolejna klasa dziedzicząca po *Osobie*. Reprezentuje pracownika uczelni, więc zamiast indeksu posiada własny numer identyfikacyjny (linia 52). Tutaj również nadpisaliśmy funkcję *drukuj()* (linia 59), aby poprawnie formatowała napisy na ekranie. Ta klasa celowo nie dziedziczy interfejsu, by pokazać różnice w strukturze.

```

66 class ListaObecnosci{
67     private:
68         Student *tabOsob[10];
69         bool tabObecnosci[10];
70         int licznik;
71         string nazwaZajec;
72         Pracownik prowadzacy;
73
74     public:
75     ListaObecnosci();
76     void ustawZajecia(string, string, string, string);
77     void drukujNazwe();
78     bool dodajOsobe(Student*);
79     bool ustawObecnosc(string, int);
80     void drukujListeObecnosci();
81     void drukujListeOsob();
82     void zmianaDanych_Edycja(int, string, string, string);
83     void zmianaDanych_Usuwanie(int);
84     int getLicznik();
85 };

```

Zarządzanie Zajęciami (Linie 66-86)

Główna klasa odpowiadająca za logikę konkretnych zajęć. Wykorzystuje tablicę wskaźników na studentów (linia 68), dzięki czemu nie kopiujemy obiektów, a jedynie na nie wskazujemy. Obok działa tablica logiczna *bool* (linia 69) przechowująca obecność.

```

88 class InterfejsUzytkownika{
89     private:
90         ListaObecnosci listy[10];
91         int ileList;
92         int aktualnaLista;
93         Student *tabOsob;
94         int iluStudentow;
95
96     public:
97     InterfejsUzytkownika();
98     void setTablicaOsob(Student*);
99     void petla();
100 };
101
102 void drukuj(IPrzedstawialny* obj){
103     cout << "\n" << obj->przedstawSie();
104 }

```

Interfejs i Funkcja Zewnętrzna (Linie 88-105)

Klasa odpowiedzialna za komunikację z użytkownikiem i menu. Zarządza tablicą, w której mieści się do 10 różnych zajęć (linia 90). Zewnętrzna funkcja *drukuj()* w linii 102 to świetny przykład użyteczności interfejsu – przyjmuje wskaźnik na dowolny obiekt zgodny z *IPrzedstawialny* i wywołuje jego metodę, nie martwiąc się o to, jakiej konkretnie klasy jest to obiekt.

```

107 ListaObecnosci::ListaObecnosci() {
108     licznik = 0;
109     nazwaZajec = "";
110     for(int i = 0; i < 10; i++) {
111         tabOsob[i] = NULL;
112         tabObecnosci[i] = false;
113     }
114 }
115
116 void ListaObecnosci::ustawZajecia(string _nazwaZajec, string
    _imie, string _nazwisko, string _nr) {
117     nazwaZajec = _nazwaZajec;
118     prowadzacy.setImie(_imie);
119     prowadzacy.setNazwisko(_nazwisko);
120     prowadzacy.setNr_Pracownika(_nr);
121 }
122
123 void ListaObecnosci::drukujNazwe() {
124     cout << nazwaZajec;
125     prowadzacy.drukuj();
126 }

```

Początkowe Ustawienia Zajęć (Linie 107-127)

Konstruktor w linii 107 pilnuje, aby nowe zajęcia zaczynały się z czystą kartą – zeruje licznik i ustawia wskaźniki na NULL, żeby program nie odwoływał się do pustej pamięci. Metoda w linii 116 pozwala nadać zajęciom nazwę i od razu przekazuje dane prowadzącego do stworzonego wcześniej obiektu klasy Pracownik, korzystając z jego wewnętrznych funkcji (setterów).

```

129 bool ListaObecnosci::dodajOsobe(Student *osoba) {
130     if(licznik < 10) {
131         tabOsob[licznik] = osoba;
132         tabObecnosci[licznik] = false;
133         licznik++;
134         return true;
135     } else {
136         cout << "\nError! Lista jest pelna.";
137         return false;
138     }
139 }

```

Zapisywanie na Zajęcia (Linie 129-140)

Funkcja dodająca studenta do wybranej listy zajęć. Najpierw upewnia się, czy jest jeszcze miejsce (linia 130). Jeśli tak, wpisuje adres studenta do tablicy (linia 131), domyślnie zaznacza go jako nieobecnego i zwraca true jako potwierdzenie sukcesu. Jeśli lista jest pełna, funkcja odrzuca akcję i zwraca false, co pozwala głównemu menu odpowiednio zareagować.

```

142 bool ListaObecnosci::ustawObecnosc(string nazwisko, int wybor) {
143     bool znaleziono = false;
144     for(int i = 0; i < licznik; i++) {
145         if(tabOsob[i]->getNazwisko() == nazwisko) {
146             if(wybor == 1) {
147                 tabObecnosci[i] = true;
148             } else {
149                 tabObecnosci[i] = false;
150             }
151             znaleziono = true;
152         }
153     }
154     return znaleziono;
155 }

```

Sprawdzanie Obecności (Linie 142-156)

Program przeszukuje dodanych studentów za pomocą pętli. W linii 145 używamy operatora ->, by dostać się do nazwiska studenta przez wskaźnik i porównać je z wpisanym tekstem. Jeśli nazwiska się zgadzają, program zmienia status ucznia na obecny (true) lub nieobecny (false) i odnotowuje, że udało się znaleźć taką osobę, co pozwala uniknąć błędów w przypadku literówek.

```

158 void ListaObecnosci::drukujListeObecnosci() {
159     if(licznik == 0) {
160         cout << "\nBrak osob na tej liscie.\n";
161         return;
162     }
163     cout << "\n###| Lista Obecnosci |###";
164     for(int i = 0; i < licznik; i++) {
165         cout << "\n" << i+1 << ": ";
166         tabOsob[i]->drukuj();
167         cout << " - ";
168         if(tabObecnosci[i]) {
169             cout << "Obecny";
170         } else {
171             cout << "Nieobecny";
172         }
173     }
174     cout << "\n";
175 }

```

Wyświetlanie Wyników (Linie 158-176)

Ta metoda najpierw sprawdza, czy na liście ktokolwiek jest – jeśli nie, od razu kończy działanie (linia 159). Następnie w pętli przechodzi przez wszystkich studentów i prosi ich obiekty o wypisanie swoich danych (linia 166). Na sam koniec, na podstawie równoległej tablicy bool, dopisuje informację tekstową o tym, czy dany student był obecny, czy nie.

```

186 void ListaObecnosci::zmianaDanych_Usuwanie(int wybor) {
187     if(wybor < licznik && wybor > -1) {
188         for(int i = wybor; i < licznik - 1; i++) {
189             tabOsob[i] = tabOsob[i+1];
190             tabObecnosci[i] = tabObecnosci[i+1];
191         }
192         tabOsob[licznik-1] = NULL;
193         tabObecnosci[licznik-1] = false;
194         licznik--;
195     } else {
196         cout << "\nError! Poza lista... ";
197     }
198 }

```

Usuwanie Osób z Zajęć (Linie 186-199)

Gdy usuniemy kogoś ze środka listy, w tablicy tworzy się "dziura". Aby temu zapobiec, program w linii 188 uruchamia pętlę, która bierze osoby znajdujące się poniżej usuniętego studenta i przesuwa je o jedno miejsce w górę. Na koniec programu musimy tylko "wyczyścić" zdublowany wpis na samym końcu (linie 192-193) i pomniejszyć główny licznik klasy.

```

217 void InterfejsUzytkownika::petla() {
218     if(tabOsob == NULL) {
219         cout << "Bład krytyczny: Interfejs nie otrzymał zrodla
220             danych!";
221         return;
222     }
223     bool CzyProgramOdpalony = true;
224     int wybor = 0;
225     string imie, nazwisko, nrIndex, nazwaZajec, nrPracownika;
226
227     do {
228         // [Wyświetlanie menu...]
229         if(aktualnaLista != -1) {
230             cout << "\n[Wybrane zajecia: ";
231             listy[aktualnaLista].drukujNazwe();
232             cout << "]\n";
233         } else {
234             cout << "\n[Nie wybrano zadnych zajec]\n";
235         }
236         cout << "\nPodaj numer: ";
237         cin >> wybor;

```

Główna Pętla Programu (Linie 217-247)

Funkcja zarządzająca całym menu widocznym na ekranie. Zabezpieczenie z linii 218 pilnuje, żeby menu się nie uruchomiło, jeśli nie otrzymało prawidłowej bazy danych od funkcji main. Cały program działa na pętli do...while, która kręci się dopóki użytkownik nie wybierze opcji wyjścia. W linii 233 na bieżąco pokazujemy, na jakich zajęciach obecnie pracujemy.

```

250     switch (wybor) {
251     case 1:
252         if(ileList < 10) {
253             cout << "\n###| Nowe Zajecia |###";
254             cout << "\nPodaj nazwe zajec: "; cin >>
                nazwaZajec;
255             cout << "\nPodaj imie prowadzacego: "; cin >>
                imie;
256             cout << "\nPodaj nazwisko prowadzacego: "; cin
                >> nazwisko;
257             cout << "\nPodaj numer pracownika: "; cin >>
                nrPracownika;
258
259             listy[ileList].ustawZajecia(nazwaZajec, imie,
                nazwisko, nrPracownika);
260             aktualnaLista = ileList;
261             ileList++;
262         } else {
263             cout << "\nError! Maksymalna ilosc zajec.";
264         }
265         break;

```

Obsługa Menu: Tworzenie Zajęć (Linie 250-265)

Kiedy użytkownik wybierze opcję numer 1, program sprawdza, czy nie osiągnięto limitu 10 zajęć. Następnie prosi o wpisanie potrzebnych danych – co ważne, zbiera od razu dane dla nowego obiektu Pracownika. Następnie przekazuje je do funkcji konfiguracyjnej z danej listy (linia 260) i od razu przełącza widok programu tak, by pracować na nowo utworzonych zajęciach.

```

290     case 3:
291         if(aktualnaLista == -1) { break; }
292         if(iluStudentow >= 10) { break; }
293         // [Pobieranie imienia, nazwiska...]
294
295         tabOsob[iluStudentow].setImie(imie);
296         tabOsob[iluStudentow].setNazwisko(nazwisko);
297         tabOsob[iluStudentow].setNr_Index(nrIndex);
298
299         if(listy[aktualnaLista].dodajOsobe(&tabOsob[
                iluStudentow])) {
300             iluStudentow++;
301         }
302         break;

```

Obsługa Menu: Nowy Student (Linie 290-313)

Ten fragment wymaga nieco więcej uwagi. Zanim w ogóle zapytamy o dane, upewniamy się, czy w ogóle mamy wybrane jakieś zajęcia i czy jest jeszcze miejsce w głównej tablicy pamięci (linia 292). Następnie budujemy obiekt studenta w głównej bazie, zapisując do niego teksty (linie 298-300). W linii 302 podajemy adres tego ucznia do konkretnych zajęć. Licznik ogólny wzrasta tylko, jeśli ta akcja faktycznie się udała.

```

358     case 5:
359         if(aktualnaLista == -1) { break; }
360         if(listy[aktualnaLista].getLicznik() == 0) { break;
                }
361
362         cout << "\nPodaj nazwisko osoby: "; cin >> nazwisko;
363         cout << "\nCzy ta osoba jest obecna? [T: 1 | N: 2]:
                "; cin >> wybor;
364         if(!listy[aktualnaLista].ustawObecnosc(nazwisko,
                wybor)) {
365             cout << "\nError! Nie znaleziono osoby o takim
                nazwisku.";
366         }
367         break;

```

Obsługa Menu: Oznaczanie (Linie 358-372)

Tutaj wprowadziliśmy podwójne zabezpieczenie. Oprócz sprawdzenia, czy wybrano zajęcia, weryfikujemy też, czy te zajęcia nie są przypadkiem puste (linia 360). Dopiero wtedy pozwalamy na edycję. W linii 364 użyliśmy wykrzyknika (!) przed funkcją. Jeśli ustawObecnosc zwróci fałsz (bo np. wpisaliśmy błędne nazwisko), wykrzyknik odwróci to na prawdę, co spowoduje uruchomienie komunikatu o błędzie.

```
394 int main() {  
395     Student tabOsob[10];  
396     InterfejsUzytkownika ui;  
397  
398     ui.setTablicaOsob(tabOsob);  
399     ui.petla();  
400  
401     return 0;  
402 }
```

Start Programu (Linie 394-403)

Funkcja `main` jest bardzo krótka i to jest jej ogromna zaleta. Zajmuje się tylko przygotowaniem fundamentów. W linii **395** rezerwujemy miejsce w pamięci na główną tablicę mogącą pomieścić 10 studentów. W linii **396** uruchamiamy nasz interfejs graficzny. Przekazujemy mu klucze (adres) do bazy danych (linia **398**), po czym włączamy nieskończoną pętlę menu (linia **399**), oddając kontrolę całkowicie użytkownikowi.